

Stealthy Enumeration of Active Directory Environments Through ADWS

 logan-goins.com/2025-02-21-stealthy-enum-adws

Logan Goins



This blog was originally published on IBM Think [here](#).

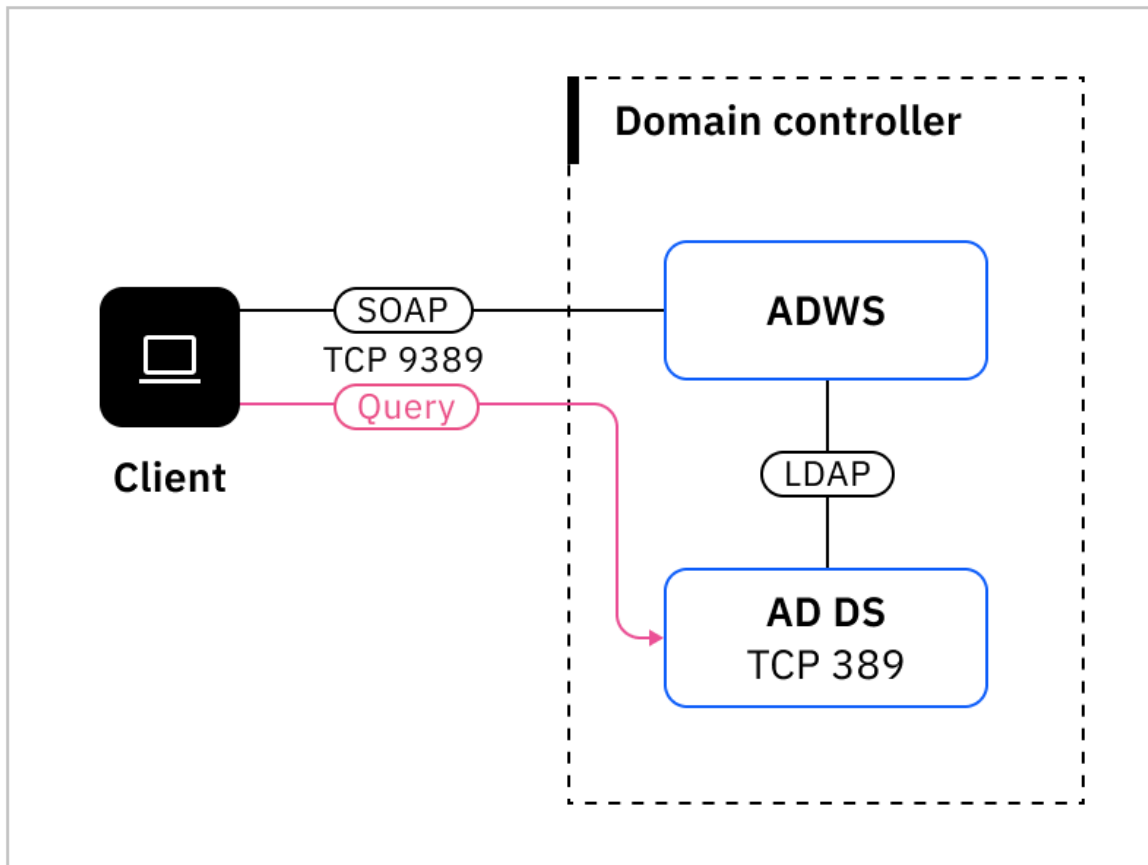
Introduction

Over time, both targeted and large-scale enumeration of Active Directory (AD) environments have become increasingly detected due to modern defensive solutions. During our internship at X-Force Red this past summer, we noticed FalconForce's SOAPHound was becoming popular for enumerating Active Directory environments. This tool brought a new perspective to Active Directory enumeration by performing collection via Active Directory Web Services (ADWS) instead of directly through Lightweight Directory Access Protocol (LDAP) as other AD enumeration tools had in the past. We were interested in expanding the use cases of this tradecraft, which eventually led us to simplify interaction with ADWS from Linux hosts through the development of a portable library written in Python and a custom tool for utilizing said library that we named SOAPy.

What is Active Directory Web Services?

ADWS is enabled by default on Active Directory Domain Controllers (DCs) on port 9389 and is utilized by a variety of Microsoft systems administration tools, such as Active Directory Administrative Center (ADAC) and the Active Directory module within PowerShell. Clients communicate with ADWS using SOAP (Simple Object Access Protocol) messages in XML format. These messages are parsed by the web service, which

then interacts with the local LDAP service on the domain controller. This allows for typical AD interaction (including both reading and writing to objects) using the AD permissions assigned to the querying user without requiring a direct bind to the LDAP service itself. Moreover, as connections are passed from the local ADWS service to LDAP, any interactions done using this mechanism are displayed as the local domain controller connecting to itself within Windows Event Logs.



ADWS houses a collection of protocols that are exposed via web service endpoints. Each endpoint has a uniquely identifying Uniform Resource Identifier (URI) and is preceded by a `net.tcp` binding type. Two authentication mechanisms are supported for interaction. Including Windows Integrated authentication for using a Windows native protocol called NNS (.NET NegotiateStream Protocol), as well as the Username/Password mechanism used for authentication over Transport Layer Security (TLS). Different endpoints provide different functionality from ADWS. For example, the Enumeration endpoint can be used to query and read LDAP data, and the Resource endpoint can be used to write LDAP data. The full list of web service endpoints is shown below.

Endpoint URI	Protocol exposed by endpoint	Authentication mechanism (see below)
net.tcp://localhost:9389/ActiveDirectoryWebServices/Windows/Resource	[WXFR], [MS-WSTIM]	Windows Integrated
net.tcp://localhost:9389/ActiveDirectoryWebServices/Windows/ResourceFactory	[MS-WSTIM]	Windows Integrated
net.tcp://localhost:9389/ActiveDirectoryWebServices/Windows/Enumeration	[WSENUM], [MS-WSDS]	Windows Integrated
net.tcp://localhost:9389/ActiveDirectoryWebServices/Windows/AccountManagement	[MS-ADCAP]	Windows Integrated
net.tcp://localhost:9389/ActiveDirectoryWebServices/Windows/TopologyManagement	[MS-ADCAP]	Windows Integrated
net.tcp://localhost:9389/ActiveDirectoryWebServices/UserName/Resource	[WXFR], [MS-WSTIM]	Username/password
net.tcp://localhost:9389/ActiveDirectoryWebServices/UserName/ResourceFactory	[MS-WSTIM]	Username/password
net.tcp://localhost:9389/ActiveDirectoryWebServices/UserName/Enumeration	[WSENUM], [MS-WSDS]	Username/password
net.tcp://localhost:9389/ActiveDirectoryWebServices/UserName/AccountManagement	[MS-ADCAP]	Username/password
net.tcp://localhost:9389/ActiveDirectoryWebServices/UserName/TopologyManagement	[MS-ADCAP]	Username/password
net.tcp://localhost:9389/ActiveDirectoryWebServices/mex	[WSMETA]	None

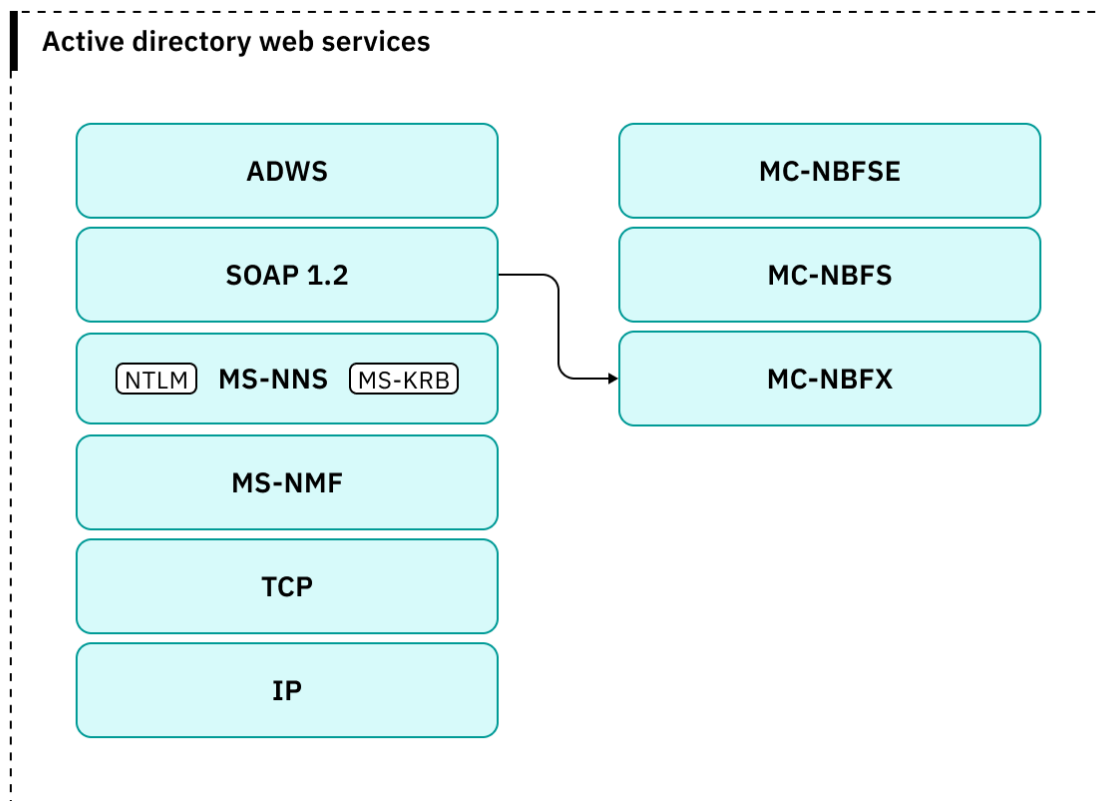
The Problem & Our Solution

Before the creation of our library, interacting with ADWS could only be done by utilizing Microsoft-built tools such as RSAT (Remote Server Administration Tools), and tools created using .NET, which essentially limited usage of the protocol to Windows hosts. Having the ability to interact with this service from a Linux host could give security professionals additional options for Active Directory interaction.

This gap motivated us to create SOAPy, a tool for interacting with LDAP over ADWS from a Linux host. Creating this tool held a variety of challenges to overcome, as the underlying protocols used to interact with ADWS had not yet been implemented in Python. The relative lack of documentation on these protocols further complicated matters and resulted in us reverse engineering them both through source code analysis and examination of packet captures.

Some of the technologies we ended up implementing in Python to successfully communicate over ADWS include NNS (.NET NegotiateStream Protocol), NMF (.NET Message Framing Protocol) and NBFSE (.NET Binary Format: SOAP Extension). These implementations with the rest of our tool totals around 5,000 lines of code. Because of the

number of relatively obscure protocol layers required to interact with LDAP over ADWS, it took several months of work before even being able to make a simple query over ADWS.



Interacting with Active Directory Web Services The first protocol layer our team was required to engineer for interacting with ADWS was NMF, the specification for this protocol can be found [here](#). This protocol defines how messages should be framed and is primarily used for framing SOAP messages. NMF includes an initial handshake used to establish the session, with the first message sent from the client being the NMF Preamble message. This message includes the mode of operation (always duplex mode in the case of ADWS), a via record, which allows us to set the designated ADWS web endpoint on the server to interact with and finally, the encoding format to use for data transfer. A code example showcasing the structure of these messages is shown in Figure 4. To our understanding, the only encoding format that is supported is NBFSE, which will be touched on later. As seen below, the via records format is always prepended with `net.tcp://`, followed by the hostname of the desired server, the port for the ADWS service and finally the specified web endpoint. When requesting data from LDAP, we want to use the Enumeration endpoint.

```
# send the preamble
NMFpreamble(
    version=(1, 0),
    mode=self._mode,
    via=f"net.tcp://{self._fqdn}:9389/ActiveDirectoryWebServices/{resource}",
    encoding=self._encoding,
).send(self._transport)
```

Following the NMF Preamble message, the client sends an NMF Upgrade Request message (0x9), requesting permission to upgrade the session using NNS authentication and start the NNS handshake. If the server permits this request it responds with an NMF Upgrade Response message (0xA).

NNS functions to provide framing for Generic Security Service Application Program Interface (GSS-API) data and utilizes Simple and Protected GSS-API Negotiation (SPNEGO) to negotiate whether to use the NTLM or Kerberos authentication protocols. Additionally, NNS also provides framing for authentication via NTLM or Kerberos. The specification for NNS can be found [here](#). The below example focuses on authentication using NTLM over NNS.

An NNS handshake is next sent by the client to begin the authentication process. It specifically includes an authentication payload containing authentication tokens, which we generate using Impacket's SPNEGO library.

```
# Fit it all into an NNS NTLMSSP_NEGOTIATE Message
# Begin authentication ( NTLMSSP_NEGOTIATE )
NNS_handshake(
    message_id=MessageID.IN_PROGRESS,
    major_version=1,
    minor_version=0,
    payload=NegTokenInit.getData(),
).send(self._sock)
```

The server then sends back an NNS NTLMSSP_Challenge message, which contains a challenge that is used to build the NTLMSSP_AUTH as a challenge-response to send back to the server for authentication. After successfully authenticating, the server then sends back a final NNS handshake message (0x15) indicating the authentication's status. Something of note is that we quickly learned that ADWS was not vulnerable to NTLM relay attacks due to message signing being required server-side.

After the NMF connection has been successfully upgraded to NNS and the client has authenticated to the server, the client sends the NMF Preamble End message (0xC), telling

the server that the preamble has been completed. The server responds with an NMF Preamble Acknowledgement message (0xB), acknowledging the preamble is finished and the client can now send data.

As mentioned earlier, data sent to the server needs to be structured in the NBFSE format, as defined by the specification here. NBFSE is used to encode or serialize SOAP data to be sent over NMF. NBFSE is an extension of NBFS (.NET Binary Format: SOAP Data Structure), which itself is an extension of NBFX (.NET Binary Format: XML Data Structure), requiring us to implement all three XML formatting specifications. NBFSE requires the usage of an in-band dictionary for data reduction procedures, but we found this requirement can be bypassed by sending messages with a blank in-band dictionary.

After implementing NBFSE, our focus shifted to understanding how a client interacts with ADWS after completing the authentication process. Originally, we wanted to query LDAP, so the first data message we implemented was the ADWS Enumeration message. This message includes the LDAP query that should be used by the server to query the local LDAP service, as well as a list of LDAP attributes that should be returned for each object. Additionally, each enumeration message defines the Enumerate Action and the Enumeration endpoint. Note that each message from this point on is a full SOAP data message; for example, an Enumeration message is shown below:

```
LDAP_QUERY_FSTRING: str = """<s:Envelope xmlns:s="http://www.w3.org/2003/05/soap-envelope"
  xmlns:a="http://www.w3.org/2005/08/addressing"
  xmlns:addata="http://schemas.microsoft.com/2008/1/ActiveDirectory/Data"
  xmlns:ad="http://schemas.microsoft.com/2008/1/ActiveDirectory"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <s:Header>
    <a:Action s:mustUnderstand="1">http://schemas.xmlsoap.org/ws/2004/09/enumeration/Enumerate</a:Action>
    <ad:instance>ldap:389</ad:instance>
    <a:MessageID>urn:uuid:{uuid}</a:MessageID>
    <a:ReplyTo>
      <a:Address>http://www.w3.org/2005/08/addressing/anonymous</a:Address>
    </a:ReplyTo>
    <a:To s:mustUnderstand="1">net.tcp://{fqdn}:9389/ActiveDirectoryWebServices/Windows/Enumeration</a:To>
  </s:Header>
  <s:Body xmlns:wsen="http://schemas.xmlsoap.org/ws/2004/09/enumeration"
    xmlns:adlq="http://schemas.microsoft.com/2008/1/ActiveDirectory/Dialect/LdapQuery">
    <wsen:Enumerate>
      <wsen:Filter Dialect="http://schemas.microsoft.com/2008/1/ActiveDirectory/Dialect/LdapQuery">
        <adlq:LdapQuery>
          <adlq:Filter>{query}</adlq:Filter>
          <adlq:BaseObject>{baseobj}</adlq:BaseObject>
          <adlq:Scope>Subtree</adlq:Scope>
        </adlq:LdapQuery>
      </wsen:Filter>
      <ad:Selection Dialect="http://schemas.microsoft.com/2008/1/ActiveDirectory/Dialect/XPath-Level-1">
        <ad:SelectionProperty>addata:distinguishedName</ad:SelectionProperty>
        <ad:SelectionProperty>addata:objectClass</ad:SelectionProperty>
        <ad:SelectionProperty>addata:description</ad:SelectionProperty>
        <ad:SelectionProperty>addata:userPrincipalName</ad:SelectionProperty>
        <ad:SelectionProperty>addata:userAccountControl</ad:SelectionProperty>
        <ad:SelectionProperty>addata:sAMAccountName</ad:SelectionProperty>
        <ad:SelectionProperty>addata:objectSID</ad:SelectionProperty>
        <ad:SelectionProperty>addata:msDS-AllowedToActOnBehalfOfOtherIdentity</ad:SelectionProperty>
      </ad:Selection>
    </wsen:Enumerate>
  </s:Body>
</s:Envelope>"""
```

Action definition

Endpoint definition

Query definition

Attribute selection

Upon receiving the Enumeration message, the server responds with a message containing a session string, called an Enumeration Context in the form of a Universally Unique Identifier (UUID). We can then use this Enumeration Context in a Pull message, to pull LDAP results from the server. The Pull message is shown below containing an appropriate Action of Pull and an Enumeration Context definition.

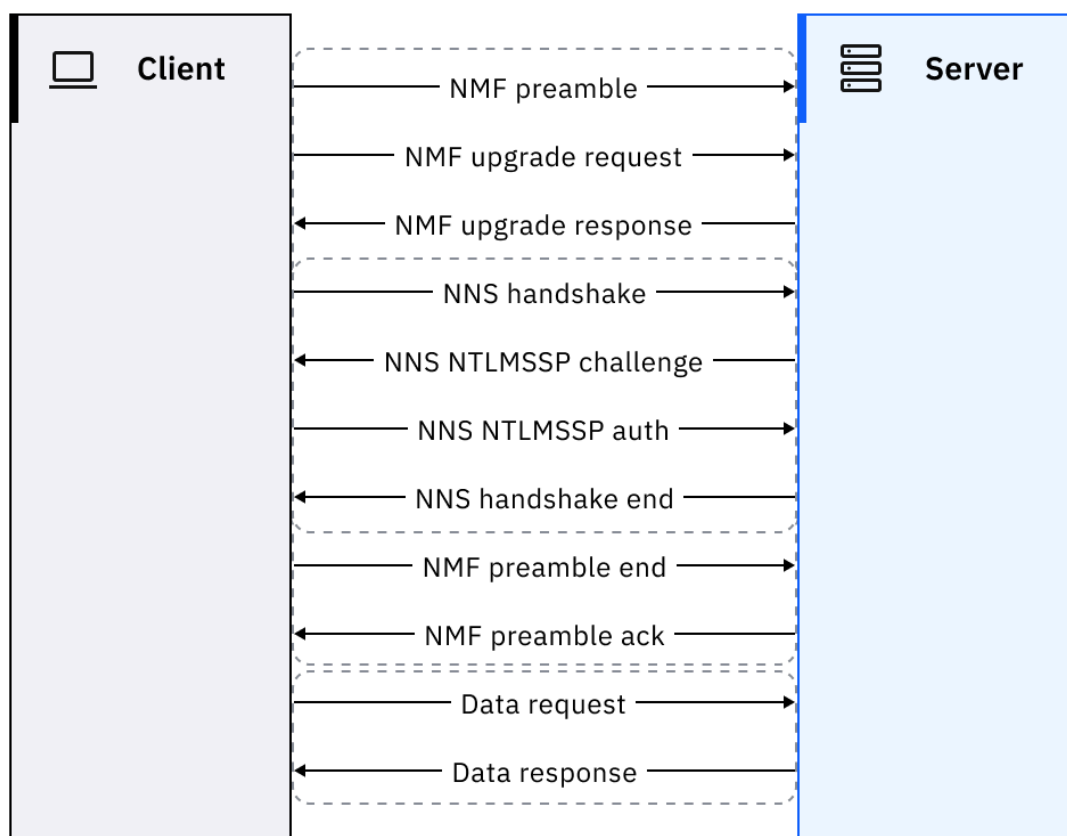
```
LDAP_PULL_FSTRING: str = """<s:Envelope xmlns:s="http://www.w3.org/2003/05/soap-envelope"
  xmlns:a="http://www.w3.org/2005/08/addressing"
  xmlns:addata="http://schemas.microsoft.com/2008/1/ActiveDirectory/Data"
  xmlns:ad="http://schemas.microsoft.com/2008/1/ActiveDirectory"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <s:Header>
    <a:Action s:mustUnderstand="1">http://schemas.xmlsoap.org/ws/2004/09/enumeration/Pull</a:Action>
    <ad:instance>ldap:389</ad:instance>
    <a:MessageID>urn:uuid:{uuid}</a:MessageID>
    <a:ReplyTo>
      <a:Address>http://www.w3.org/2005/08/addressing/anonymous</a:Address>
    </a:ReplyTo>
    <a:To s:mustUnderstand="1">net.tcp://{fqdn}:9389/ActiveDirectoryWebServices/Windows/Enumeration</a:To>
  </s:Header>
  <s:Body xmlns:wsen="http://schemas.xmlsoap.org/ws/2004/09/enumeration">
    <wsen:Pull>
      <wsen:EnumerationContext>{enum_ctx}</wsen:EnumerationContext>
      <wsen:MaxElements>256</wsen:MaxElements>
    </wsen:Pull>
  </s:Body>
</s:Envelope>"""
```

Action definition

Enumeration context definition

After this message has been sent to the server, the server will respond with LDAP information in SOAP format, which can then be further parsed by the receiving client.

The full message interaction between client and server is shown below.



SOAPy

SOAPy is a Python tool we've created that uses these underlying protocol libraries to perform LDAP reconnaissance and modification actions against remote ADWS instances. It includes a collection of pre-built queries used for common AD reconnaissance actions such as enumerating accounts with the `servicePrincipalName` attribute set, and identification of accounts configured for constrained and unconstrained delegation. SOAPy also includes a flag for custom-built queries of the operator's choosing, as well as the option to write to the `msDs-AllowedToActOnBehalfOfOtherIdentity` attribute on LDAP objects for exploiting Resource-Based Constrained Delegation (RBCD).

Most of the common `impacket` example script usage conventions carry over into SOAPy as our original goal for this project was to create a tool that could overlay effectively with the `Impacket` suite. Utilizing the `Impacket` suite made interacting with well-documented Active Directory authentication protocols such as NTLM and Kerberos quite easy, but as the current `Impacket` project did not support NNS, NMF, etc. we extended the project with the additional protocols we implemented in SOAPy.

As an example, SOAPy can be used to retrieve user accounts with the `servicePrincipalName` attribute set by passing in the `-spns` flag:


```
root@kali:~/SoaPy# python3 soa.py lab.lan/jdoe:'P@ssw0rd'@192.168.1.2 --spns

SOAPv

[*] Connecting to 192.168.1.2 for Enumeration
[+] Object Found: user
    SAMAccountName: mssql_svc
    objectSid: S-1-5-21-4081171455-1080558525-301034555-1108
    objectClass: top person organizationalPerson user
    userPrincipalName: mssql_svc@lab.lan
    distinguishedName: CN=MSSQLSvc,CN=Users,DC=lab,DC=lan
    userAccountControl: 590336 flags: NORMAL_ACCOUNT, DONT_EXPIRE_PASSWORD, TRUSTED_FOR_DELEGATION
```

In the above demo, a single result is returned – the mssql_svc user. Currently, only a default subset of attributes is displayed for returned objects, but in the future, we would like to allow the operator to customize specific attributes to be returned by the query.

SOAPy is available as open-source tooling on the official IBM X-Force Red GitHub page, at <https://github.com/xforcered/SOAPy>.

Development Methodology

Gathering logs from ADWS to recreate these protocols proved to be difficult, as the only logging mechanisms identified to gather information about the protocol were Windows Communication Foundation (WCF) logging (enabled through the ADWS service configuration file) and .NET logging. Most of the development process was done via observation of network traffic generated by PowerShell's Active Directory module, review of WCF logging and the reading of each protocol specification in the protocol stack.

WCF logging can be enabled by modifying

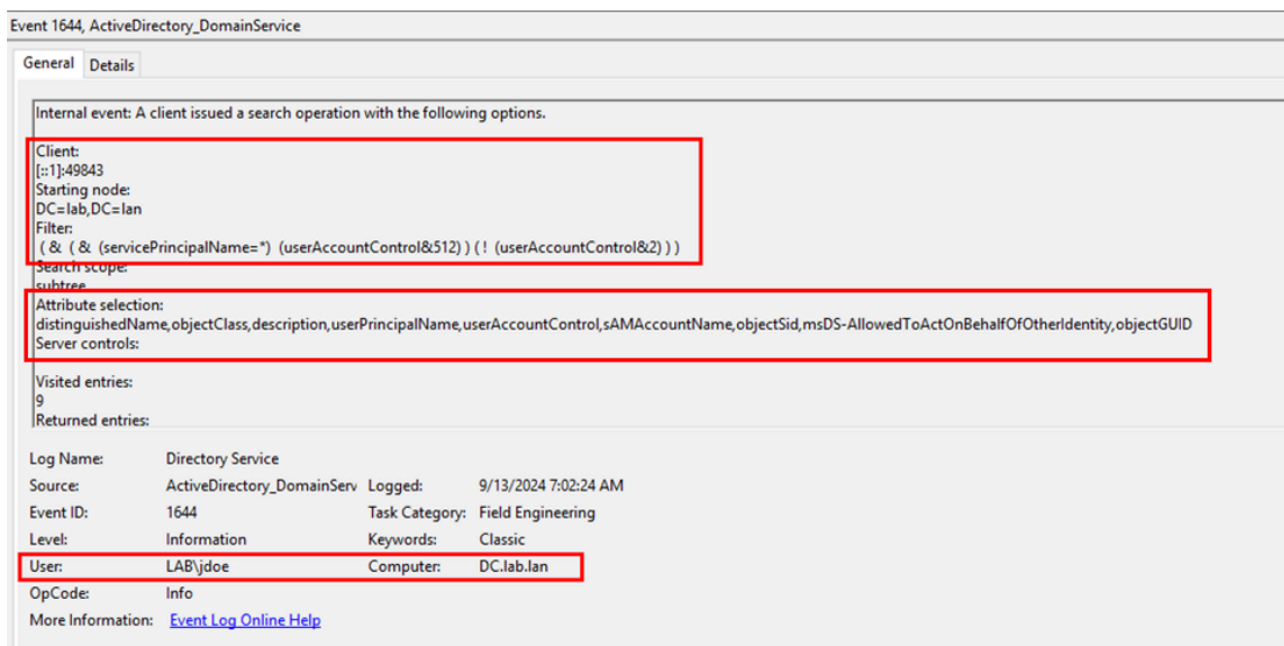
C:\Windows\ADWS\Microsoft.ActiveDirectory.WebServices.exe.config. Specifics of configuration are detailed in official Microsoft documentation.

Detection Considerations

LDAP logging is an enumeration detection method used to gather additional information on details of LDAP interactions in Active Directory environments. Some of the important information returned from the logging includes the client address that initiated the query, the computer from which the query originates, the LDAP filter string used, attributes selected for return and finally the user context used for authentication to the LDAP server.

As an example, the following screenshot is of the Windows Event Viewer with LDAP logging enabled after performing Active Directory enumeration with SOAPy.

Information on enabling LDAP logging can be found [here](#).



Commonplace LDAP recon detection methods still apply when detecting enumeration from SOAPy. Although the client is not directly interacting with the LDAP service, the interaction from ADWS doesn't obscure everything useful. Malicious indicators still are passed to the LDAP service from ADWS, including the LDAP Filter, the attribute selection and the originating user account that provided authentication. The above screenshot displays a common suspicious LDAP query used to enumerate Kerberoastable accounts. Previously implemented LDAP detections will still trigger from this event, although as the query was made against ADWS the log will show a source computer of a local domain controller. The log will also show a low-privilege user in the Domain Users group having performed a query from the DC because of the indirect LDAP access through ADWS, which is unusual in any other scenario given the permissions required for access to the DC. Additionally, System Access Control List (SACL) canaries are still effective at logging access to specific objects while using SOAPy, quickly alerting defenders to suspicious activity.

While the detection of enumeration from SOAPy is similar to the detection of direct LDAP enumeration, additional complexity arises when finding the source of the enumeration as part of incident response procedures. This is due to the originating computer and IP address in the event always being the DC. One way of finding the potential source of enumeration would be to correlate the user performing the enumeration with the active sessions in the environment. While this can be effective if the user context being used for operating the post-exploitation capability is the same as the user context performing enumeration, this may not always be a completely effective approach. This is due to the possibility of the post-exploitation capability being used to proxy traffic into the environment and provide authentication using stolen credentials.

With these considerations in mind, typical alerts for LDAP-based reconnaissance should still be effective at alerting defenders to the presence of anomalous behavior in the environment and can provide a solid Indicator-of-Compromise (IOC) for the user object

used to perform the query. However, they may require additional review to determine the source host of the action.

Next Steps

We intend to maintain our codebase and continue to improve it while adding new features and quality-of-life improvements including additional options for fine-grained attribute collection, custom attribute writing and ADCS certificate enumeration. Integrating our underlying libraries and SOAPy into Impacket in the form of a GitHub Pull Request is still a goal for us. We feel our backend interaction for interacting with NNS, NMF, etc. might be useful for future tooling developers looking to interact with any other services utilizing these protocols, mostly because to our knowledge Python code for interacting with these protocols didn't exist previously.

Conclusion

Active Directory Web Services, or ADWS, has been a default-enabled service on Domain Controllers since Windows Server 2008, and it allows us to interact with LDAP, making queries on our behalf and proxying our queries. We noticed interaction with ADWS was previously not possible through a Linux host, which motivated us to create SOAPy. SOAPy came with its own difficulties during development, requiring us to create custom protocol implementations with little assistance from Microsoft specifications. SOAPy also has its own accompanying detection considerations, being a significantly stealthier method of LDAP enumeration instead of interacting directly with the LDAP service.

We hope that SOAPy lays the foundation for interacting with ADWS over a Linux host, or any service which utilizes the underlying protocols required for interaction. It is a major goal to get our code merged into Impacket, helping ensure that our code is widespread and accessible while pushing the community to use our project as a jumping off point for further development.